TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN HỌC

MÃ HÓA DỮ LIỆU VÀ CHUỖI KHỐI BLOCKCHAIN

Đề Tài:

Ứng Dụng Hyperledger Fabric Trong Precision Farming

SVTH: Phạm Nguyễn Thanh Vy_2274802011036
Lâm Xuân Phú_2274802010666
GVHD: TS. Huỳnh Thái Học

TP. Hồ Chí Minh – năm 2025

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

PHIẾU NHẬN XÉT SINH VIÊN

CỦA GIẢNG VIÊN HƯỚNG DẪN

1.Họ và tên sinh viên: Lâm Xuân Phú_2274802010666

Phạm Nguyễn Thanh Vy_2274802011036

Lớp: 242_71ITDS40503_01

Ngành: Mạng máy tính và Truyền thông dữ liệu

Giảng viên hướng dẫn: TS. Huỳnh Thái Học

2.Nhận xét:

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí minh, ngày tháng năm 2025

Giảng viên hướng dẫn

(Ký và ghi rõ họ và tên)

LỜI CẢM ƠN

Trước tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Quý Thầy Cô Khoa Công Nghệ Thông Tin – Đại Học Văn Lang đã tận tình hướng dẫn, truyền đạt kiến thức và hỗ trợ chúng em trong suốt quá trình học tập và thực hiện đồ án này. Những chỉ dẫn quý báu từ Thầy Cô đã giúp chúng em hoàn thiện kiến thức chuyên môn và kỹ năng cần thiết để hoàn thành tốt nhiệm vụ được giao.

Đặc biệt, chúng em gửi lời tri ân sâu sắc đến Thầy Huỳnh Thái Học, người đã luôn đồng hành, động viên và hỗ trợ chúng em trong suốt quá trình nghiên cứu và hoàn thiện đồ án. Những góp ý chân thành và sự tận tâm của Thầy đã giúp chúng em hoàn thiện tốt hơn đề tài này.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí minh, ngày tháng năm 2025

Sinh viên thực hiện

(Ký và ghi rõ họ và tên)

(Nhóm Trưởng)



Lâm Xuân Phú

MỤC LỤC

CHƯƠNG I: MỞ ĐẦU 6
1. Lý do chọn đề tài 6
2. Đối tượng, phạm vi nghiên cứu 6
2.1. Đối tượng nghiên cứu 6
2.2. Phạm vi nghiên cứu 7
CHƯƠNG II: LÝ THUYẾT 8
2.1. Hyperledger Fabric trong Blockchain 8
2.2 Các composement của Blockchain 8
2.3. Các công cụ hỗ trợ Hyperledger 9
2.4. Hợp đồng thông minh (Smart Contract) 10
2.5. Các thành phần chính trong Hyperledger Fabric 10
2.6. Cách hoạt động của Hyperledger Fabric 12
2.7. Ưu điểm và nhược điểm của Hyperledger Fabric 12
CHƯƠNG III: THỰC NGHIỆM 14
3.1. Bối cảnh 14
3.2. Mục tiêu 14
3.3. Giải pháp đề xuất 14
3.4. Kiểm soát Sức khỏe Cây trồng Dựa trên Mô hình Dự đoán 15
3.5. An toàn Thực phẩm và Bảo mật Chuỗi Cung ứng Phân Tán 15
3.6. Kiến trúc Hệ thống 15
3.7. Chạy Demo 16
3.7.1. Code 16
3.7.2. Sản phẩm 22
CHƯƠNG IV: TỔNG KẾT 24
TÀI LIỆU THAM KHẢO 25

DANH MỤC HÌNH ẢNH

Hình 1: Cách hoạt động của Smart Contract 10
Hình 2: Luồng giao dịch trong Hyperledger Fabric 12
Hình 3 16
Hình 4 17
Hình 5 17
Hình 6 18
Hình 7 18
Hình 8 19
Hình 9 19
Hình 10 19
Hình 11 20
Hình 12 20
Hình 13 20
Hình 14 21
Hình 15 21
Hình 16: Giao diện người dùng 22
Hình 17 23
Hình 18 23

CHƯƠNG I: MỞ ĐẦU

1. Lý do chọn đề tài

Nông nghiệp là một thành phần quan trọng trong sự phát triển về kinh tế của một quốc gia. Nhận thức được tầm quan trọng của nông nghiệp, các quốc gia đã từ lâu áp dụng công nghệ hiện đại như IoT, AI vào nông nghiệp như một điều thiết yếu để giúp giải quyết các vấn đề trong nông nghiệp cũng như là tối ưu hóa tối đa sản lượng trong nông nghiệp. Các hệ thống nông nghiệp sử dụng công nghệ hiện đại này còn được gọi là Precision Farming. Tuy nhiên, gần đây đã xuất hiện các tình trạng có liên quan đến quản lý và bảo mật dữ liệu trong nông nghiệp như sau: "Dữ liệu thu thập từ cảm biến IoT (độ ẩm đất, nhiệt độ môi trường, chất dinh dưỡng,...) có thể bị sai lệch do lỗi thiết bị hoặc bị thao túng; Dữ liệu trong nông nghiệp bị chia sẻ bừa bãi khiến cho ai cũng có quyền truy cập; Các hệ thống cảm biến, thiết bị điều khiển tự động có thể bị tấn công DDoS thông qua các lỗ hổng bảo mật phần mềm. Nắm bắt được những vấn đề ấy, hệ thống Hyperledger Fabric đã bắt đầu được áp dụng vào Precision Farming nhằm giúp nâng cao khả năng bảo mật và quản lý dữ liệu trong nông nghiệp. Đây chính là một đề tài rất phù hợp với chủ đề của bộ môn "Mã hóa dữ liệu và Chuỗi khối (Blockchain).Chúng em hy vọng rằng thông qua đồ án này, mọi người sẽ hiểu rõ hơn về cách hoạt động của hệ thống Hyperledger Fabric trong Precision Farming, các vấn đề liên quan đến lỗ hổng bảo mật trong hệ thống Precision Farming cũng như các kiến thức về bảo mật mạng để chuẩn bị tốt hơn cho công việc thực tế sau này.

Tuy nhiên, một số vấn đề về bảo mật và quản lý dữ liệu trong nông nghiệp xuất hiện như:

- Dữ liệu thu thập từ cảm biến IoT có thể bị sai lệch hoặc bị thao túng.
- Dữ liệu nông nghiệp bị chia sẻ bừa bãi, mất quyền kiểm soát.
- Các hệ thống IoT có thể bị tấn công DDoS thông qua lỗ hổng bảo mật.

Do đó, Hyperledger Fabric đã được áp dụng vào Precision Farming để nâng cao bảo mật và quản lý dữ liệu.

2. Đối tượng, phạm vi nghiên cứu

2.1. Đối tượng nghiên cứu

• Công nghệ blockchain, cụ thể là Hyperledger Fabric, là một nền tảng blockchain permissioned được sử dụng trong dự án để tạo ra hệ thống lưu trữ dữ liệu phi tập trung. Nhằm đảm bảo tính bảo mật và minh bạch của dữ liệu trong nông nghiệp.

• Tích hợp Hyperledger Fabric với Precision Farming: Ứng dụng blockchain trong quản lý dữ liệu, truy xuất nguồn gốc, tự động hóa giao dịch, bảo mật dữ liệu trong nông nghiệp.

2.2. Phạm vi nghiên cứu

• Tập trung xây dựng và triển khai smart contracts (chaincode) để tự động hóa các giao dịch và hợp đồng giữa các bên liên quan trong hệ sinh thái nông nghiệp.

• Giám sát mô hình Precision Farming thông qua các thiết bị IoT để giảm bớt chi phí sản xuất và tránh lãng phí bằng cách tối ưu hóa các sản lượng đầu vào, đồng

thời tăng năng suất thông qua việc cung cấp dữ liệu chính xác cho quyết định canh tác.

CHƯƠNG II: LÝ THUYẾT

2.1. Hyperledger Fabric trong Blockchain

Giới thiệu về Blockchain

- Blockchain là công nghệ cho phép sử dụng sổ cái phân tán để theo dõi mọi thay đổi hoặc giao dịch diễn ra trong mạng lưới. Hệ thống sổ cái này được phân cấp và được lưu trữ trên mọi hệ thống có sự xuất hiện của blockchain.

2.2 Các composement của Blockchain

- Hệ thống mạng blockchain sử dụng một mô-đun phần mềm được gọi là smart contracts (hay còn được gọi là chaincode) cho phép kiểm soát quyền truy cập vào sổ cái phân tán, tự động hóa các quy trình, kiểm soát các khía cạnh của giao dịch và thực hiện một số hành động nhất định nếu đáp ứng các điều kiện đã xác định. Các thay đổi hoặc giao dịch diễn ra trong mạng được phê chuẩn hoặc từ chối thông qua sự đồng thuận giữa tất cả các thành viên của mạng. Sau khi được phê chuẩn, giao dịch sau đó được ghi vào sổ cái. Dữ liệu hoặc giao dịch diễn ra trong mạng và được nhập vào sổ cái sẽ được mã hóa, lưu trữ vào một khối và sau đó trở thành không thể sửa đổi (hoặc không thể thay đổi).

Giới thiệu về Hyperledger Fabric

- Hyperledger Fabric là một khuôn khổ blockchain riêng tư và là một trong nhiều dự án thuộc nền tảng Blockchain Hyperledger. Khuôn khổ này được sử dụng làm nền tảng để phát triển các ứng dụng, mạng lưới dựa trên Blockchain, Fabric được thiết kế để tạo ra các blockchain riêng tư có thể được sử dụng trong một tổ chức hoặc nhóm các tổ chức liên kết với các nền tảng blockchain khác. Fabric sử dụng một số tính năng chính như là:

  • **Quyền riêng tư:** Fabric yêu cầu tất cả máy tính trong network của nó đều phải được xác định. Các thành viên trong network này được fabric hỗ trợ phải xác định bản thân thông qua Nhà cung cấp dịch vụ thành viên (MSP). Đây là thứ được gọi là tư cách thành viên "được cấp phép". Việc duy trì quyền riêng tư của dữ liệu là điều cần thiết đối với nhiều ngành và chỉ riêng khía cạnh này đã khiến Fabric trở thành một lựa chọn hấp dẫn.

  • **Kênh (Channels):** Fabric cung cấp khả năng phân vùng sổ cái thành "kênh", nơi các thành viên của mạng có thể tạo một tập hợp giao dịch riêng biệt mà mạng lớn hơn không nhìn thấy. Điều này cho phép tách biệt dữ liệu nhạy cảm hơn khỏi các nút không yêu cầu quyền truy cập.

Kênh trong Hyperledger Fabric có thể được định nghĩa là một thực thể ligic đại diện cho một nhóm gồm hai hoặc nhiều thành viên tham gia mạng lưới Blockchain nhằm mục đích thực hiện các giao dịch riêng tư và bảo mật giữa họ. Do đó, bất kỳ giao dịch nào trên mạng lưới Hyperledger Fabric Blockchain đều được thực hiện trong kênh. Sau đây là các khối xây dựng chính của một kênh:

- Tổ chức như các thành viên.
- Một hoặc nhiều đồng nghiệp neo cho mỗi tổ chức thành viên.
- Chính sách kênh.
- Sổ cái chia sẻ phân tán (riêng tư cho các thành viên của kênh).
- Ứng dụng Chaincode.
- Đặt hàng các nút dịch vụ

• **Khả năng mở rộng:** Một đặc điểm hấp dẫn khác của Fabric đối với các doanh nghiệp lớn là network do fabric cung cấp có khả năng mở rộng cực lớn. Giống như các cách triển khai khác, số lượng nút tham gia vào mạng có thể mở rộng nhanh chóng; nhưng hệ thống vẫn có khả năng xử lý lượng dữ liệu lớn với một tập hợp tài nguyên nhỏ hơn.

• **Tính mô-đun:** Fabric được thiết kế để cho phép các thành phần riêng biệt được thêm vào và triển khai tại các thời điểm khác nhau. Nhiều thành phần tùy chọn có thể được bỏ qua hoàn toàn hoặc được giới thiệu sau mà không ảnh hưởng đến chức năng trong hệ thống. Tính năng này nhằm mục đích trao cho công ty quyền kiểm soát những gì cần thiết hoặc không để dễ triển khai. Một số thành phần được coi là mô-đun hoặc "cắm và chạy", bao gồm phương pháp đạt được sự đồng thuận, dịch vụ thành viên để nhận dạng, bản thân kho sổ cái , API truy cập cụ thể và tích hợp mã chuỗi .

Với mục tiêu phát triển một khuôn khổ có khả năng mở rộng, được cấp phép và bảo mật cao được hỗ trợ bởi công nghệ sổ cái phân tán của blockchain, Hyperledger Fabric đã khuấy động thế giới mạng lưới doanh nghiệp.

2.3. Các công cụ hỗ trợ Hyperledger

• **Hyperledger Composer:** là một bộ công cụ mã nguồn mở giúp tăng tốc phát triển ứng dụng trên Hyperledger Fabric. Tuy nhiên, Hyperledger Composer đã không còn được sử dụng từ tháng 8 năm 2019 do chức năng được cung cấp bởi Composer đã được đưa vào chính Fabric.

• **Hyperledger Quilt:** là một công cụ Hyperledger được thiết kế để tạo điều kiện thuận lợi cho việc chuyển giao các giao dịch giữa các mạng blockchain khác nhau.

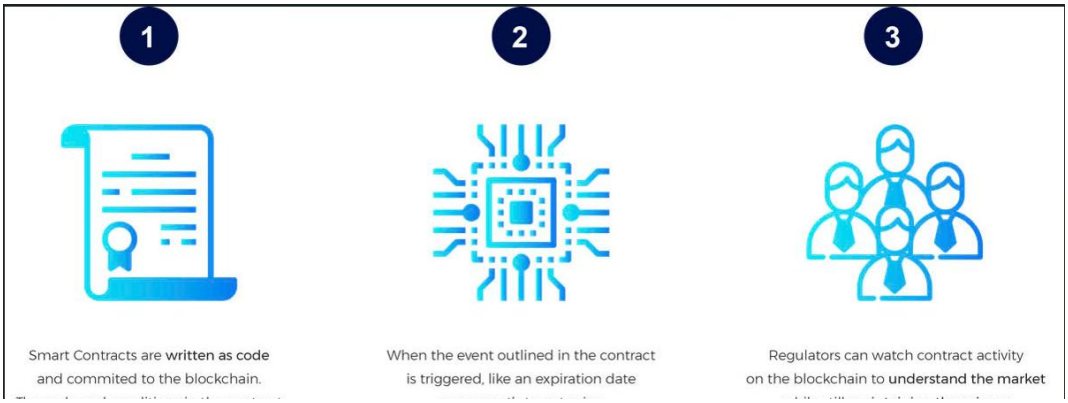
• **Hyperledger Explorer:** được phát triển chủ yếu để truy cập mọi thông tin trong mạng blockchain; truy vấn các khối, truy cập dữ liệu giao dịch, thành viên và các nút tương ứng, thông tin về chính mạng lưới và hầu như mọi thứ thuộc về blockchain cụ thể đó.

2.4. Hợp đồng thông minh (Smart Contract)

- Smart contract (còn được hiểu theo cách khác là Chaincode) cùng với sổ cái là những khía cạnh quan trọng nhất của mạng Fabric. Trong blockchain, smart contract đóng vai trò hỗ trợ giao dịch tiền, hàng hóa hoặc bất kỳ tài sản nào khác được biểu thị trong mạng. Các hợp đồng này có thể đặt giá, đưa ra các quy tắc, phác thảo thỏa thuận và thậm chí ủy quyền các hành động cần thực hiện khi đáp ứng được các yêu cầu nhất định. Những điều kiện này có thể vô hạn và đơn giản như thanh toán một số tiền nhất định, được hoàn lại tiền nếu hàng hóa không được giao hoặc cung cấp biên lai kỹ thuật số. Các hợp đồng này diễn ra trong blockchain dưới dạng mã thực thi.

- Những hợp đồng thông minh này đóng một vai trò quan trọng trong Fabric, hoạt động như những hợp đồng theo nghĩa đen. Smart Contract giúp đơn giản hóa đáng kể quá trình giao dịch bằng cách tự động hóa các quy trình và xử lý các giao dịch thanh toán.

2.5. Các thành phần chính trong Hyperledger Fabric



Hình 1: Cách hoạt động của Smart Contract

Peer (Nodes)

- Peer (hay là Node) là một nút mạng ngang hàng trong hệ thống Hyperledger Fabric. Nó có tính năng lưu trữ sổ cái, thực thi các hợp đồng thông minh và xác thực các giao dịch trong Blockchain. Có 3 loại Peer:

• **Endorser Peer:** Chịu trách nhiệm thực thi hợp đồng thông minh (Smart Contract) và mô phỏng giao dịch trước khi chúng được ghi vào blockchain. Sau khi thực thi, nó sẽ tạo chữ

ký xác nhận và gửi lại kết quả cho Client App. Chỉ những giao dịch có đủ số lượng chữ ký Endorser hợp lệ mới được gửi đến Ordering Service.

• **Anchor Peer:** Là điểm giao tiếp chính giữa các tổ chức khác nhau trong một mạng blockchain. Được sử dụng để đồng bộ dữ liệu giữa các công ty trong cùng một kênh (channel). Các Peer trong tổ chức khác sẽ sử dụng Anchor Peer để biết được thông tin của tổ chức đó.

• **General Peer:** Chỉ có nhiệm vụ lưu trữ sổ cái và xác thực giao dịch, nhưng không thực thi hợp đồng thông minh như Endorser Peer. Nhận khối giao dịch từ Ordering Service và cập nhật dữ liệu vào sổ cái.

Hyperledger Fabric Certificate Authority

- Là thành phần quản lý danh tính và cấp chứng chỉ số trong mạng blockchain của Hyperledger Fabric. Nó đóng vai trò quan trọng trong việc xác thực danh tính của các thành viên trong hệ thống và đảm bảo tính bảo mật của các giao dịch.

- Chức năng chính của CA là cấp chứng chỉ số để xác thực danh tính của người dùng, Peer và Client. Nếu đã cấp chứng chỉ thì đương nhiên CA cũng có chức năng quản lý danh tính hoặc thu hồi chứng chỉ nếu thấy một thực thể không còn hợp lệ trong hệ thống Blockchain. Ngoài ra, CA còn hỗ trợ cấp quyền và kiểm soát truy cập nếu được kết hợp với Membership Service Provider (MSP).

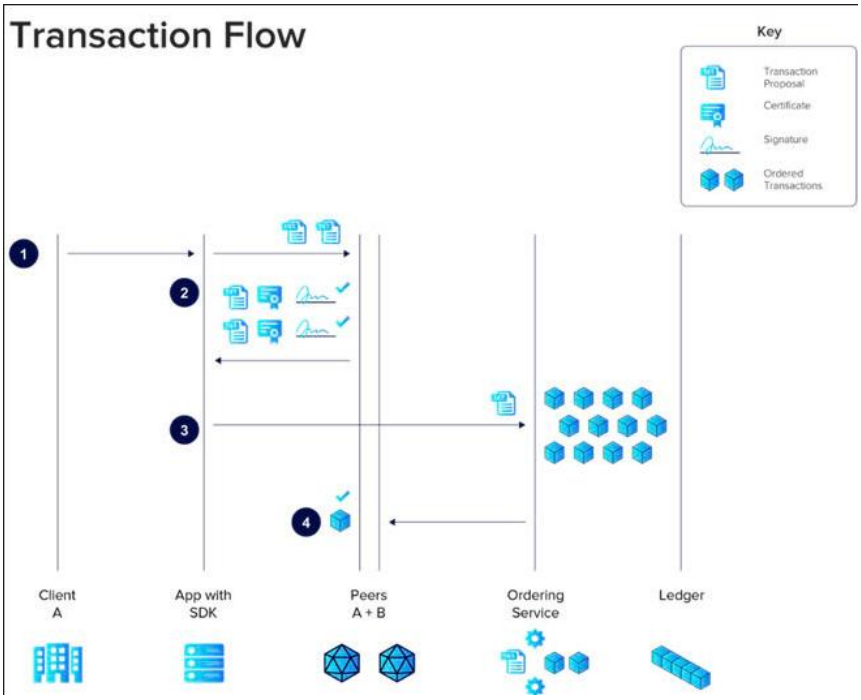
Sổ cái (Ledger)

- Ledger trong Hyperledger Fabric là sổ cái kỹ thuật số lưu trữ tất cả các giao dịch diễn ra trong blockchain. Nó giúp đảm bảo tính minh bạch, bảo mật và không thể thay đổi của dữ liệu trong hệ thống. Cấu trúc của Ledger bao gồm hai thành phần chính:

• **World State:** Lưu trữ trạng thái hiện tại của tất cả các tài sản trên Blockchain. Có thể sử dụng CouchDB hoặc LevelDB để lưu trữ và được cập nhật khi có giao dịch hợp lệ.

• **Blockchain Ledger:** Là một danh sách bất biến dùng để lưu trữ tất cả các giao dịch đã diễn ra từ trước đến nay. Mỗi giao dịch trong blockchain sẽ tạo một khối (block) mới, liên kết với block trước đó bằng mã băm (hash).

2.6. Cách hoạt động của Hyperledger Fabric



Hình 2: Luồng giao dịch trong Hyperledger Fabric

• **Bước 1:** Client tạo một đề xuất giao dịch và gửi nó đến Application SDK để xử lý. Đề xuất này sẽ bao gồm thông tin về giao dịch, chẳng hạn như ID tài sản, giá trị, hành động.

• **Bước 2:** Ứng dụng khách hàng (hoặc SDK) sẽ gửi Đề xuất đến các Endorser Peers (Peers A + B). Mỗi Peer Nodes xác nhận giao dịch bằng cách kiểm tra giao dịch có hợp lệ hay không. Nếu hợp lệ, Peers sẽ ký xác nhận (endorsement) và trả về kết quả.

• **Bước 3:** Sau khi được xác nhận và trả về, giao dịch sau đó được gửi đến Ordering Service. Cơ chế Ordering Service sẽ sử dụng consensus (sự đồng thuận) để sắp xếp giao dịch vào một khối trong sổ cái.

• **Bước 4:** Sau khi được sắp xếp, khối giao dịch được lưu trữ sẽ được gửi đến tất cả các Peers như một phần của channel. Đây là bước xác thực cuối cùng trước khi thực hiện thay đổi sổ cái (Việc thay đổi sổ cái là bước quan trọng để đảm bảo hệ thống ghi nhận các giao dịch mới một cách bất biến, minh bạch và đáng tin cậy).

2.7. Ưu điểm và nhược điểm của Hyperledger Fabric

Ưu điểm:

• **Quyền riêng tư và bảo mật cao:** Hyperledger Fabric hỗ trợ các kênh riêng tư (private channels) giúp đảm bảo chỉ những thành viên được ủy quyền mới có thể truy cập vào dữ

liệu. Bên cạnh đó Hyperledger Fabric còn cho phép sử dụng Cơ quan cấp chứng chỉ (Certificate Authority - CA) để xác thực danh tính.

• **Cấu trúc mô-đun linh hoạt:** Hỗ trợ nhiều loại cơ chế đồng thuận (consensus mechanisms) như Kafka, Raft, giúp tối ưu hiệu suất theo từng trường hợp sử dụng. Có thể tùy chỉnh Smart Contract (Chaincode) bằng nhiều ngôn ngữ lập trình như Go, Java, Node.js thay vì chỉ sử dụng Solidity như Ethereum.

• **Hiệu suất cao và mở rộng tốt:** Không cần đào (mining) như Bitcoin hay Ethereum, giúp tăng tốc độ xử lý giao dịch. Cấu trúc chia tách vai trò của Peer Nodes (Endorser, Orderer, Committer) giúp giảm tải và tăng khả năng mở rộng.

• **Giao dịch có thể kiểm soát:** Doanh nghiệp có thể đặt quy tắc giao dịch (Endorsement Policy) để kiểm soát việc thực thi hợp đồng thông minh. Hỗ trợ quản lý quyền truy cập (Access Control) dựa trên vai trò của từng thành viên.

• **Tổ chức quản trị rõ ràng:** Không giống như các blockchain công khai (Ethereum, Bitcoin), Hyperledger Fabric là Permissioned Blockchain, giúp các doanh nghiệp kiểm soát ai có thể tham gia vào mạng.

Nhược điểm:

• **Triển khai phức tạp:** So với các blockchain công khai như Ethereum, việc cài đặt và cấu hình Hyperledger Fabric khó hơn do có nhiều thành phần như Orderer, Peer, CA, Channel.

• **Cộng đồng nhỏ hơn Ethereum, Bitcoin:** Hyperledger Fabric không phổ biến bằng Ethereum hoặc Bitcoin, dẫn đến số lượng developer hỗ trợ ít hơn và khó tìm tài liệu hơn.

• **Không hỗ trợ token gốc:** Không giống như Ethereum có ETH, Hyperledger Fabric không hỗ trợ token gốc, nên nếu doanh nghiệp muốn tạo token, họ phải phát triển riêng.

• **Phụ thuộc vào tổ chức vận hành:** Vì đây là Permissioned Blockchain, mạng chỉ hoạt động khi có các tổ chức tham gia duy trì. Nếu một số tổ chức rời bỏ, hệ thống có thể bị gián đoạn.

CHƯƠNG III: THỰC NGHIỆM

3.1. Bối cảnh

Nông nghiệp chính xác (Precision Farming) đang trở thành xu hướng quan trọng để tối ưu hóa sản xuất, giảm lãng phí và đảm bảo an toàn thực phẩm. Tuy nhiên, các thách thức như giám sát chính xác điều kiện canh tác, phát hiện vấn đề sức khỏe cây trồng, và đảm bảo an toàn chuỗi cung ứng vẫn còn tồn tại. Ứng dụng ‘Health-Smart Secure Farming (HSSF)‘ được phát triển để giải quyết những vấn đề này bằng cách tích hợp công nghệ Internet of Things (IoT), học máy (Machine Learning), và blockchain Hyperledger Fabric.

3.2. Mục tiêu

Mục tiêu của HSSF là đạt được "nông nghiệp thông minh và an toàn"thông qua ba điều kiện chính:

- Giám sát chính xác điều kiện canh tác, thời tiết, và đất đai.
- Phát hiện và kiểm soát các vấn đề liên quan đến sức khỏe cây trồng và hiệu quả sản xuất.
- Đảm bảo an toàn thực phẩm và bảo mật phân tán trong chuỗi cung ứng.

3.3. Giải pháp đề xuất

Công nghệ sử dụng: Thu thập dữ liệu thời gian thực từ các cảm biến đo nhiệt độ, độ ẩm, độ pH đất, và độ ẩm không khí. Dữ liệu này được truyền qua LPWAN đến hệ thống trung tâm.

Blockchain Prototype: Sử dụng một hệ thống blockchain nội bộ (Python) để lưu trữ thông tin cảm biến, dữ liệu nông dân, thông tin ruộng và thiết bị, giúp minh bạch và không thể thay đổi dữ liệu (dựa trên blockchain_code.py).

AI/Machine Learning: Dựa trên dữ liệu lịch sử cảm biến, mô hình AI sẽ học cách cây trồng phát triển và đề xuất chiến lược canh tác tối ưu.

Lợi ích:

- Giảm chi phí sản xuất và lãng phí đầu vào (nước, phân bón, thuốc trừ sâu).
- Tăng năng suất bằng cách cung cấp dữ liệu chính xác để hỗ trợ quyết định.
- Ví dụ: Đối với cây nho, hệ thống có thể phát hiện sương giá vào mùa xuân hoặc mùa thu sớm để tránh thiệt hại cho nụ hoa và chồi non.

• Học mô hình tăng trưởng cây trồng từ dữ liệu cảm biến, tích hợp qua LPWAN để đưa ra các chiến lược canh tác thông minh.

3.4. Kiểm soát Sức khỏe Cây trồng Dựa trên Mô hình Dự đoán

Chi tiết:

• Tưới tiêu chính xác: Tránh tình trạng tưới quá nhiều hoặc quá ít, vì cả hai đều có thể làm tăng nguy cơ bệnh tật.

• Bón phân chính xác: Sử dụng phân bón nitơ ở mức hợp lý để tránh độc tính amoniac và bệnh do nấm.

• Sử dụng thuốc trừ sâu sinh học: Áp dụng giải pháp thân thiện môi trường dựa trên hợp tác với Đại học Taitung.

Kết quả: Cải thiện sức khỏe cây trồng, giảm thiểu tác động môi trường và nâng cao chất lượng sản phẩm.

3.5. An toàn Thực phẩm và Bảo mật Chuỗi Cung ứng Phân Tán

Công nghệ Blockchain:

Mô hình minh họa bằng Python Blockchain (blockchain_code.py + app.py): Cho phép thêm dữ liệu nông dân, ruộng, cảm biến và truy xuất chuỗi khối qua API Flask.

Triển vọng mở rộng: Có thể triển khai trên Hyperledger Fabric để đạt tính bảo mật, phân quyền và hiệu năng cao hơn.

Ứng dụng:

• Theo dõi nguồn gốc sản phẩm từ nông trại đến người tiêu dùng.

• Dữ liệu như “ruộng”, “nông dân” và “cảm biến” được ghi nhận bằng các API như **/add_farmer**, **/add_field**, **/add_sensor** và lưu trên blockchain.

• Ví dụ: Chuỗi cung ứng cá ngừ quản lý bằng Hyperledger Fabric đã chứng minh được tính minh bạch và bảo mật dữ liệu.

3.6. Kiến trúc Hệ thống

Công nghệ Hỗ trợ

Blockchain Layer:

- *Prototype:* Blockchain Python nội bộ cho thử nghiệm và mô phỏng.
- *Production:* Hyperledger Fabric sử dụng chaincode viết bằng Go/Node.js.

Data Layer: Hệ thống lưu trữ dữ liệu cảm biến từ các thiết bị IoT và cập nhật lên blockchain.

Application Layer: Flask Web Server cung cấp các RESTful API để thêm dữ liệu và truy vấn chuỗi khối (app.py).

Deployment Layer:

- **Kubernetes:** Triển khai ứng dụng, container hoá các thành phần (API, blockchain node, chaincode).
- **Docker:** Sử dụng để xây dựng và triển khai nhanh các dịch vụ.

IoT/Edge Devices: Cảm biến được kết nối qua LPWAN và đồng bộ dữ liệu lên server

3.7. Chạy Demo

Sản phẩm sử dụng 4 file để chạy cùng nhau và thành quả là tạo ra một trang web có thể đáp ứng được các mục tiêu trên.

3.7.1. Code

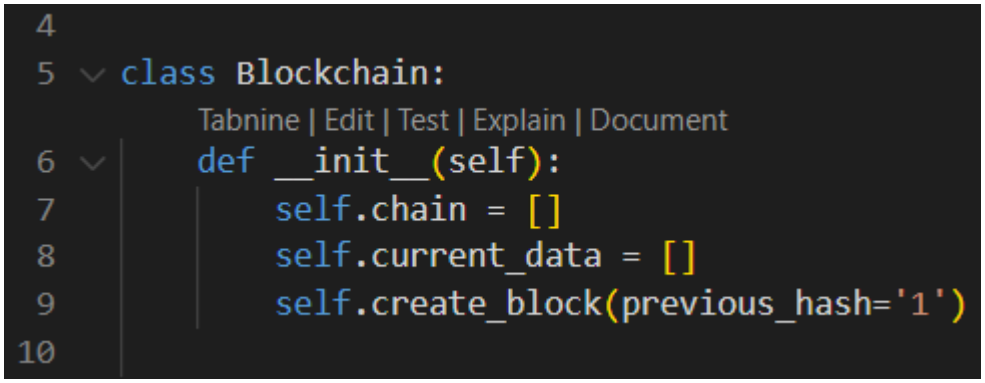
- File **index.html** và **style.css:** đây là 2 file cần thiết để tạo ra giao diện của trang web.
- File **blockchain_code.py:** Đây là file chứa những quy tắc bất biến cốt lõi của chuỗi blockchain.

  Phân tích:

```
1  import hashlib
2  import json
3  from time import time
4
```

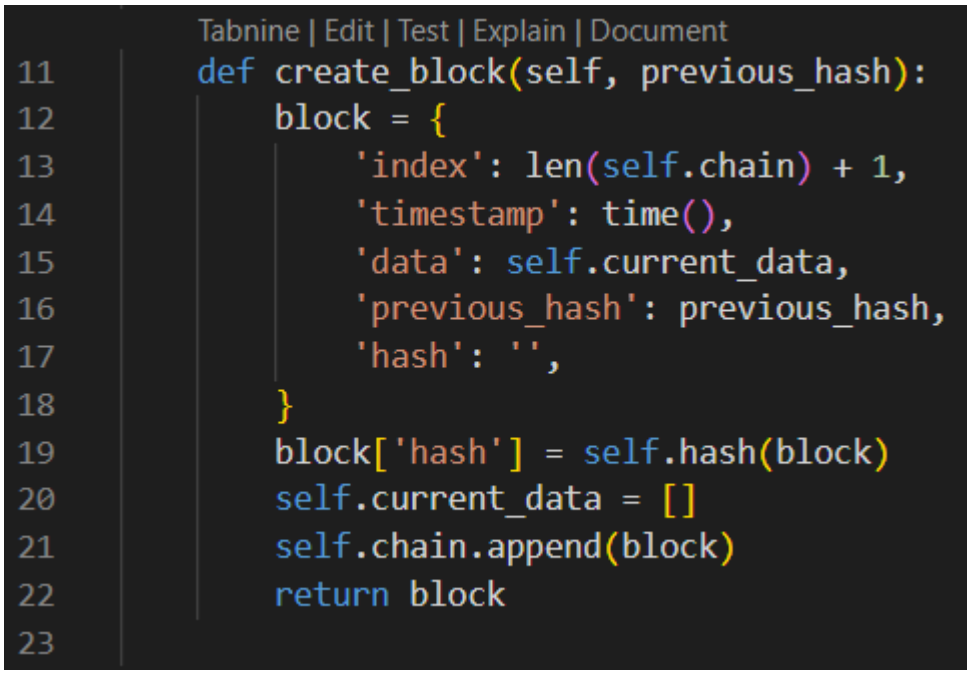
Hình 3

- Trước khi xây dựng bất kỳ chuỗi nào, chương trình cần những công cụ nền tảng: **hashlib**: dùng để tạo **hàm băm SHA-256**, giúp mã hóa dữ liệu thành chuỗi ký tự duy nhất; **json**: hỗ trợ chuẩn hóa dữ liệu thành định dạng JSON để dễ lưu trữ, dễ truyền tải; **time**: cung cấp dấu thời gian (timestamp) – đánh dấu thời điểm mà dữ liệu được ghi lại.
- Đây là ba yếu tố cốt lõi của blockchain nhằm tạo nên một trang web an toàn, minh bạch và không thể thay đổi dữ liệu bên trong.



Hình 4

- Lớp Blockchain là khung xương sống của cả hệ thống. Trong hàm khởi tạo __init__, chuỗi blockchain được tạo dưới dạng một danh sách rỗng, sau đó thêm **block đầu tiên** – hay còn gọi là **khối khởi nguyên (Genesis Block)**. Khối này được xem là nền móng của toàn bộ chuỗi.



Hình 5

- Mỗi lần có một sự kiện diễn ra (một người nông dân đăng ký, một cảm biến gửi số liệu,...) hệ thống sẽ tạo một block mới để lưu giữ sự kiện đó. Block mới này sẽ chứa:
  - **Index**: vị trí trong chuỗi.
  - **Timestamp**: thời điểm ghi nhận.
  - **Data**: nội dung chính – có thể là thông tin nông dân, cánh đồng, cảm biến...

- **Previous_hash**: mã băm của block trước đó – đảm bảo sự liên kết chuỗi.
- **Hash**: mã băm của chính block này – sau khi block được tạo, hàm hash_block() được gọi để mã hóa toàn bộ nội dung và gán vào đây.

- Bằng cách mã hóa nội dung + liên kết mã của block trước, blockchain tạo ra một **chuỗi ký ức không thể sửa đổi** – nếu bạn thay đổi một block ở giữa, toàn bộ chuỗi sau đó sẽ bị ảnh hưởng.

```
Tabnine | Edit | Test | Explain | Document
def add_data(self, data_type, data):
    entry = {'type': data_type, 'data': data}
    self.current_data.append(entry)
    return self.create_block(self.get_last_block()['hash'])

Tabnine | Edit | Test | Explain | Document
def get_last_block(self):
    return self.chain[-1]
```

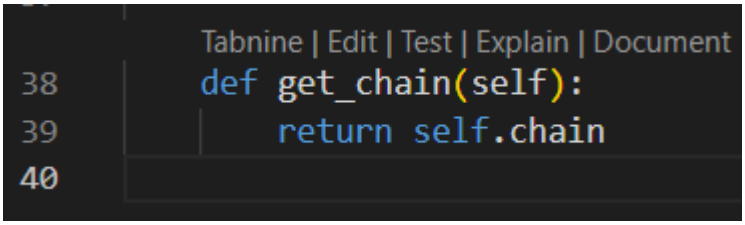
Hình 6

- Đây là cầu nối giữa dữ liệu người dùng gửi lên (qua app.py) và cấu trúc bên trong blockchain. Hàm add_data gói gọn dữ liệu lại thành một dict với hai phần: **type**-loại dữ liệu; **content**-nội dung cụ thể. Sau đó nó gọi create_block() với dữ liệu đó, kèm theo mã băm của block trước, để nối block mới vào chuỗi.

```
Tabnine | Edit | Test | Explain | Document
32  def hash(self, block):
33      block_copy = block.copy()
34      block_copy['hash'] = ''
35      encoded_block = json.dumps(block_copy, sort_keys=True).encode()
36      return hashlib.sha256(encoded_block).hexdigest()
37
```

Hình 7

- Hàm hash_block lấy nội dung của block (trừ phần hash), sắp xếp lại để tránh sai lệch thứ tự, rồi mã hóa thành chuỗi nhị phân và tính toán hàm băm SHA-256. Kết quả là một chuỗi ký tự duy nhất, đại diện cho toàn bộ nội dung block. Nếu chỉ một chi tiết nhỏ bị thay đổi (ví dụ: số độ ẩm từ 40 thành 41), hàm băm sẽ hoàn toàn khác, khiến chuỗi mất tính liên tục – một cách **tự động phát hiện và ngăn chặn chỉnh sửa trái phép.**

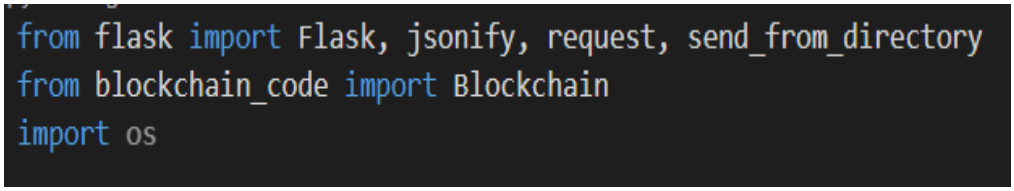


Hình 8

- Hàm này cho phép client (hoặc người dùng) xem lại toàn bộ chuỗi dữ liệu từ Genesis Block cho đến block gần nhất. Mỗi block là một **chứng cứ rõ ràng** để nói lên rằng sự kiện đó đã xảy ra – vào thời điểm đó, với nội dung chính xác ấy.

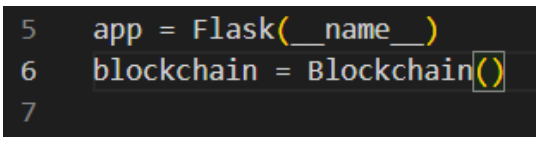
- File **app.py:** Đây là file dùng để chạy server web ứng dụng quản lý nông nghiệp thông minh. File này đóng vai trò là backend của ứng dụng và sử dụng công nghệ blockchain để xử lý các yêu cầu từ client, lưu trữ và trả về dữ liệu thông qua blockchain.

  Phân tích:



Hình 9

- Dòng đầu tiên của chương trình nhập khẩu các thành phần cần thiết để chạy trang web này. **Flask** là bộ khung chính để xây dựng server web; **jsonify** dùng để trả dữ liệu về dưới dạng JSON; **request** cho phép truy cập vào dữ liệu mà client gửi lên; **send_from_directory** dùng để gửi file tĩnh như HTML hoặc CSS từ thư mục hiện tại. Ngoài ra, **os** cũng được nạp vào phòng trường hợp thao tác với hệ thống tệp.
- Từ **blockchain_code**, ta nhập vào lớp **Blockchain**, nơi định nghĩa cách tạo và xử lý chuỗi blockchain. Sau đó, Flask được khởi tạo thông qua:



Hình 10

- Đây là hai biến cực kỳ quan trọng: **app** đại diện cho ứng dụng Flask đang chạy, còn **blockchain** là đối tượng lưu giữ toàn bộ dữ liệu hệ thống dưới dạng các block liên kết với nhau bằng mã băm.

```
Tabnine | Edit | Test | Explain | Document
@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

Tabnine | Edit | Test | Explain | Document
@app.route('/style.css')
def style():
    return send_from_directory('.', 'style.css')

```

Hình 11

- Đây là hai hàm nhỏ nhưng không thể thiếu của trang web. Khi người dùng mở trình duyệt và gõ địa chỉ http://localhost:8080, hệ thống sẽ gửi lại tệp index.html – là giao diện chính của hệ thống. Cùng với đó, file style.css cũng được gửi theo để định dạng màu sắc, bố cục, kích thước chữ, làm cho trải nghiệm người dùng trở nên thân thiện và dễ hiểu hơn.

```
Tabnine | Edit | Test | Explain | Document
@app.route('/add_farmer', methods=['POST'])
def add_farmer():
    data = request.get_json()
    block = blockchain.add_data('farmer', data)
    return jsonify({"message": "Farmer added", "block": block}), 201

```

Hình 12

- Khi người dùng gửi thông tin của một người nông dân (ví dụ: tên, địa chỉ, loại cây trồng...), hệ thống sẽ nhận lấy JSON, gọi đến phương thức add_data() và thêm dữ liệu này vào một block mới với nhãn là 'farmer'. Block mới này được nối vào chuỗi blockchain, bảo đảm rằng thông tin đó **không thể bị sửa đổi sau khi được ghi nhận**.

```
Tabnine | Edit | Test | Explain | Document
@app.route('/add_field', methods=['POST'])
def add_field():
    data = request.get_json()
    block = blockchain.add_data('field', data)
    return jsonify({"message": "Field added", "block": block}), 201

```

Hình 13

- Tương tự, đây là nơi lưu trữ thông tin về cánh đồng: diện tích, vị trí GPS, loại đất, hay vùng gieo trồng. Dữ liệu được mã hóa và gắn nhãn là 'field', giúp việc phân loại và truy xuất sau này trở nên rõ ràng và dễ dàng hơn.

```
    Tabnine | Edit | Test | Explain | Document
28  @app.route('/add_sensor', methods=['POST'])
29  def add_sensor():
30      data = request.get_json()
31      block = blockchain.add_data('sensor', data)
32      return jsonify({"message": "Sensor added", "block": block}), 201
33
```

Hình 14

- Trong hệ thống nông nghiệp hiện đại, cảm biến đóng vai trò cực kỳ quan trọng trong việc giám sát điều kiện môi trường. API này tiếp nhận dữ liệu từ các thiết bị như cảm biến độ ẩm, nhiệt độ, ánh sáng… và lưu lại chúng một cách chính xác, kèm theo dấu thời gian và mã băm. Chính điều này giúp đảm bảo rằng mọi dữ liệu đo đạc từ hiện trường đều được ghi lại **minh bạch và không thể giả mạo**.

```
    Tabnine | Edit | Test | Explain | Document
34  @app.route('/get_chain', methods=['GET'])
35  def get_chain():
36      chain = blockchain.get_chain()
37      return jsonify(chain), 200
38
39  if __name__ == '__main__':
40      app.run(port=8080, debug=True)
41
```

Hình 15

- Hàm **get_chain** cung cấp một cái nhìn tổng thể về toàn bộ chuỗi blockchain của hệ thống. Khi truy cập vào địa chỉ /get_chain, người dùng sẽ nhận được một JSON chứa danh sách tất cả các block đã được ghi lại – mỗi block là một minh chứng cho sự kiện thực tế đã xảy ra: ai đã đăng ký, cánh đồng nào được thêm vào, cảm biến nào đã gửi dữ liệu. Nó đóng vai trò như một nhật ký điện tử không thể bị xóa, một quyển nhật ký nơi thông tin được bảo vệ khỏi mọi tác động chỉnh sửa.
- Cuối cùng, file được thiết lập để chạy trên **cổng 8080** – một cổng thường được sử dụng cho ứng dụng web nội bộ. Chế độ debug=True giúp nhà phát triển có

thể theo dõi lỗi trong quá trình thử nghiệm. Điều này đặc biệt quan trọng trong giai đoạn phát triển, kiểm thử hoặc demo ứng dụng.

- Tổng kết:

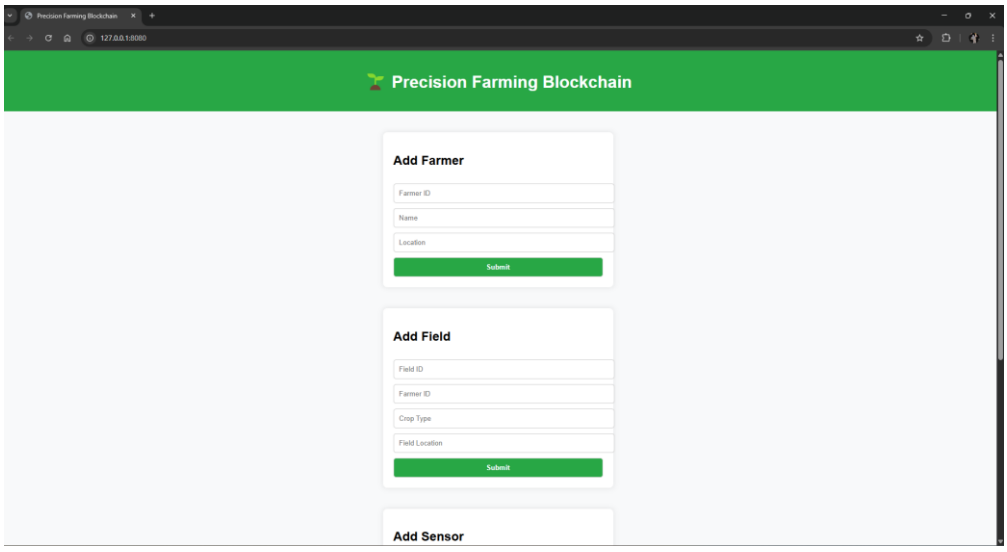
- app.py giống như một chiếc cầu nối – nơi người dùng có thể gửi thông tin, và hệ thống sẽ xử lý, ghi nhận và phản hồi lại một cách rõ ràng.
- Blockchain_code.py là nơi từng khối dữ liệu được sinh ra, mã hóa, liên kết và bảo vệ. Các dữ liệu bên trong được đảm bảo không bị lưu trữ rời rạc và được bảo vệ bằng thuật toán để không ai có thể thay đổi lịch sử mà không để lại hậu quả.

3.7.2. Sản phẩm

Sau khi xây dựng hoàn tất các hàm và class cần thiết để tạo và xử lý các block trong chuỗi blockchain, ứng dụng được triển khai bằng framework Flask của Python. Một số điểm đáng chú ý trong quá trình triển khai:

Ứng dụng tạo các endpoint RESTful cho phép thực hiện các thao tác như:

- Gửi dữ liệu cảm biến từ nông trại (ví dụ: nhiệt độ, độ ẩm, ánh sáng).
- Tạo block mới từ dữ liệu nhận được và thêm vào chuỗi blockchain.
- Truy xuất toàn bộ chuỗi blockchain.

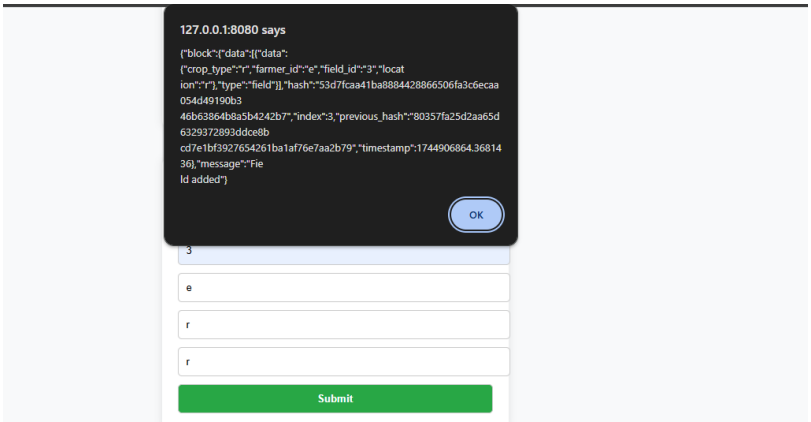


Hình 16: Giao diện người dùng

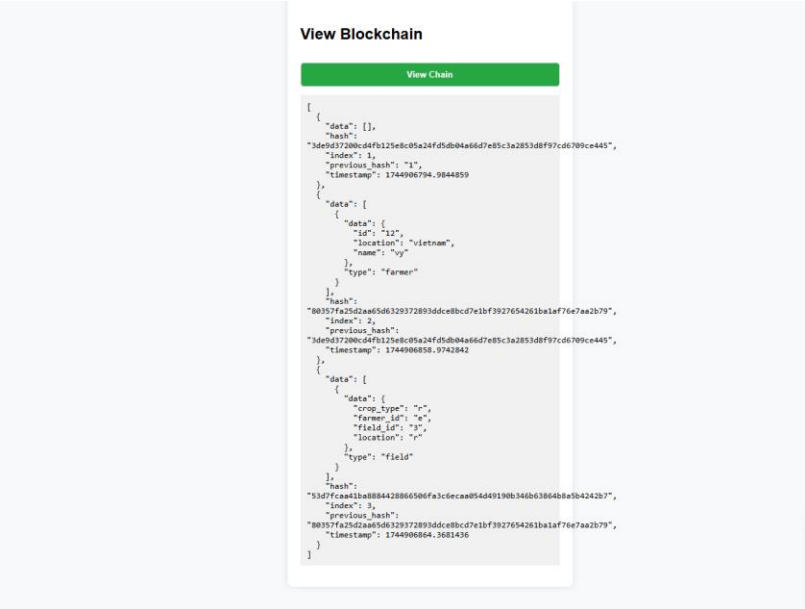
Giao diện người dùng được xây dựng đơn giản để hỗ trợ việc tương tác với hệ thống blockchain một cách trực quan. Một số đặc điểm của giao diện:

- **Trang chủ:** Cung cấp lựa chọn để gửi dữ liệu cảm biến và xem chuỗi blockchain.
- **Biểu mẫu gửi dữ liệu:** Người dùng có thể nhập dữ liệu như ID nông dân, nhiệt độ, độ ẩm,... và gửi trực tiếp đến server.
- **Hiển thị kết quả:** Sau khi gửi dữ liệu, hệ thống phản hồi lại bằng thông tin về block đã tạo và các block hiện có trong chuỗi.
- **Tương thích trình duyệt:** Giao diện hoạt động tốt trên trình duyệt web hiện đại, thuận tiện cho việc tích hợp vào các dashboard giám sát.

- Giao diện nhập dữ liệu và kết quả phản hồi từ server



Hình 17



Hình 18

CHƯƠNG IV: TỔNG KẾT

Trong đồ án này, chúng em đã triển khai thành công một mô hình ứng dụng blockchain đơn giản sử dụng ngôn ngữ lập trình Python và framework Flask, nhằm minh họa cách thức lưu trữ và xác thực dữ liệu một cách an toàn và minh bạch trong lĩnh vực nông nghiệp.

Các nội dung chính đã được thực hiện bao gồm:

- Xây dựng cấu trúc chuỗi khối (blockchain): Tạo class Block và Blockchain có khả năng khởi tạo block mới, tính toán hash, xác thực block trước, và quản lý chuỗi các block.
- Tạo API server: Sử dụng Flask để cung cấp các API phục vụ việc gửi dữ liệu cảm biến, tạo block mới và truy xuất toàn bộ chuỗi blockchain dưới dạng JSON.
- Thiết kế giao diện người dùng đơn giản: Giao diện web cho phép người dùng nhập dữ liệu và xem kết quả xử lý, góp phần minh bạch hóa quá trình vận hành và giúp dễ dàng kiểm tra, giám sát.

Mô hình đã minh chứng khả năng ứng dụng của blockchain vào các hệ thống thu thập và xác thực dữ liệu trong thực tế, đặc biệt là trong nông nghiệp thông minh – nơi mà độ tin cậy và minh bạch của dữ liệu đóng vai trò rất quan trọng.

Tuy là mô hình thử nghiệm đơn giản, hệ thống vẫn mở ra nhiều hướng phát triển trong tương lai như:

- Tích hợp IoT để thu thập dữ liệu cảm biến tự động.
- Sử dụng AI để phân tích dữ liệu từ blockchain.
- Mở rộng mạng lưới blockchain theo mô hình multi-node và multi-org nhằm tăng tính bảo mật và phân tán.

TÀI LIỆU THAM KHẢO

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf
2. Antonopoulos, A. M. (2017). Mastering Bitcoin: Programming the Open Blockchain. O'Reilly Media.
3. Hyperledger Fabric Documentation. https://hyperledger-fabric.readthedocs.io
4. Flask Documentation. https://flask.palletsprojects.com
5. GitHub - Simple Blockchain in Python: https://github.com/dvf/blockchain
6. Khanna, A., & Kaur, S. (2019). Evolution of Internet of Things (IoT) and its significant impact in the field of Precision Agriculture. Computers and Electronics in Agriculture, Elsevier.
7. Zhang, Y., et al. (2020). Blockchain-based trusted data sharing among intelligent vehicles using consortium blockchain. IEEE Transactions on Intelligent Transportation Systems.
8. Hương, P.T. & Nam, T.H. (2021). Ứng dụng Blockchain trong truy xuất nguồn gốc nông sản. Tạp chí Công nghệ Thông tin và Truyền thông Việt Nam.